

Unidad 5

DML

DML

2

Contenido:

- ▶ INSERT
- ▶ UPDATE
- ▶ DELETE

INSERT

3

- ▶ **INSERT:** comando para añadir registros a una tabla.
 - ▶ Deben cumplirse las restricciones que apliquen a los campos de la tabla
- ▶ Sintaxis:

```
INSERT INTO tbl_name () VALUES();
```

```
INSERT INTO tbl_name (col1,col2) VALUES(15,col1*2);
```

```
pero: INSERT INTO tbl_name (col1,col2) VALUES(col2*2,15); ¡¡error!!
```

INSERT

4

- ▶ **INSERT SELECT:** Se pueden insertar registros en una tabla a partir del resultado de una select .
- ▶ Sintaxis:

```
INSERT INTO nombre_tabla_destino (columna1, columna2, ...)  
SELECT columna1, columna2, ...  
FROM nombre_tabla_origen  
WHERE condiciones;
```

- Las columnas que devuelve la select debe coincidir en número y tipo con las columnas de la tabla en la que se inserta
- Si la tabla destino tiene un campo auto_increment, se puede omitir en la select
- Se pueden utilizar join, funciones de agregación, etc en la consulta select

- ▶ **CREATE TABLE ... AS SELECT...**
- ▶ Este comando permite crear una tabla a partir del resultado de una consulta SELECT. Se copia tanto la estructura como los datos.
- ▶ Esta forma de crear tablas se suele utilizar para:
 - Crear copias de seguridad de datos históricos.
 - Optimizar consultas creando tablas temporales con datos prefiltrados.
 - Crear tablas de reportes a partir de datos procesados.
- ▶ Sintaxis:

```
CREATE TABLE nueva_tabla AS  
SELECT columna1, columna2, ...  
FROM tabla_origen  
WHERE condiciones;
```

- ▶ **Consideraciones al utilizar** CREATE TABLE ... AS SELECT...
- ▶ No se copian claves primarias ni índices. Para añadir las restricciones se tiene que hacer una modificación (alter) de la tabla.
- ▶ No se copian los 'auto_increment', se copian los valores pero no la propiedad.
- ▶ La estructura de la nueva tabla se define con la SELECT, los nombres de las columnas se pueden modificar utilizando alias en

- ▶ **MULTI-ROW INSERT:** Se utiliza para insertar varias filas en un mismo comando INSERT
 - ▶ Es más eficiente que realizar múltiples INSERT individuales porque reduce la cantidad de llamadas a la base de datos.
- ▶ Sintaxis

```
INSERT INTO tabla (columna1, columna2, ...) VALUES  
(valor1_1, valor1_2, ...),  
(valor2_1, valor2_2, ...),  
(valor3_1, valor3_2, ...);
```

- ▶ **Carga masiva:** Habitualmente es necesario realizar cargas masivas en tablas de la base de datos desde archivos que contienen la información a cargar de forma estructurada.
 - ▶ Tienen las líneas con formato de registro: campos y separadores
 - ▶ Son archivos de texto plano
 - ▶ La estructura de las líneas del fichero corresponde exactamente con la estructura de la tabla que se quiere cargar.
- ▶ Los SGBD disponen de herramientas para realizar dichas cargas de forma más eficiente que utilizando un 'insert'.
- ▶ En MySQL se puede utilizar para esta tarea:
 - ▶ LOAD DATA INFILE
 - ▶ MYSQLIMPORT

- ▶ **LOAD DATA INFILE:** Habitualmente es necesario realizar cargas masivas en tablas de la base de datos desde archivos que contienen la información a cargar de forma estructurada.

- ▶ **MYSQLIMPORT:** Habitualmente es necesario realizar cargas masivas en tablas de la base de datos desde archivos que contienen la información a cargar de forma estructurada.

UPDATE

11

- ▶ **UPDATE:** Para actualizar uno o varios campos en una tabla.
 - ▶ **SET :** establece el nuevo valor para la columna en la que se aplique.
 - ▶ **WHERE:** Se aplica el filtro de los registros a los que aplicar el update.

```
UPDATE alumno SET curso = curso + 1;
```

```
UPDATE alumno SET curso = 2025, nombre = 'Antonio';
```

```
UPDATE alumno SET curso = 2025, nombre = 'Antonio' WHERE dni = '123123D' ;
```

DELETE

12

- ▶ **DELETE:** Para eliminar registros.
 - ▶ **FROM:** tabla o tablas de la que se quiere eliminar registros
 - ▶ **WHERE:** condición que deben cumplir los registros que van a ser eliminados.

```
DELETE FROM alumno WHERE dni = '123123D'
```